

Parallelization of K-Means Clustering

OpenMP and CUDA Optimization

Prof. Moreno Marzolla

Leonardo Billi

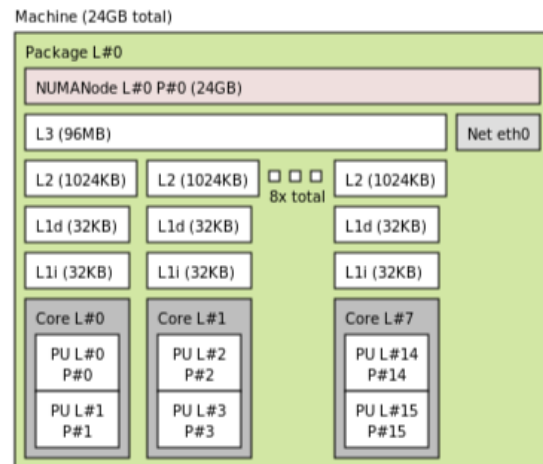
April 17, 2026

1 Hardware Architecture

1.1 CPU

The experiments were performed on a machine equipped with a single AMD Ryzen 7 7800X3D processor.

Component	Specification
Model	AMD Ryzen 7 7800X3D
Base frequency	4.2 GHz
Physical cores	8
Logical threads	16 (SMT)
L1 cache	64 KB (I+D)
L2 cache	1 MB
L3 cache	96 MB (3D V-Cache)
RAM	24 GB
NUMA nodes	1 (UMA)
OS	64-bit x64



The machine features a single NUMA node, meaning every core accesses RAM with identical latency (UMA). The 96 MB L3 cache is a direct consequence of AMD’s 3D V-Cache technology and is particularly relevant for memory-bound workloads such as K-Means. The 16 logical threads are provided by SMT, mapping two hardware threads onto each physical core. As discussed later, SMT yields a measurable speedup improvement over the 8-core baseline.

1.2 GPU

Model	VRAM	Arch.	GPC	SM	cores	L2	Bus	Driver
RTX 4060 Ti	16 GB GDDR6	AD106	3	34	4352	32 MB	128-bit	565.90

2 General Assumptions and Experimental Setup

Given the large space of possible combinations (N , K , D , Iterations), the assumptions made are as follows.

Fixed parameters.

I assume $N \gg K$ and $N \gg D$, accordingly, I fix $K = 8$ and $D = 40$, varying only N to study performance.

Fixed-iteration mode.

To remove variability introduced by Lloyd's algorithm convergence, I compile with `-DMAX_ITER_FIXED=150`, forcing exactly 150 iterations regardless of convergence. This is justified by the observation that the program reads every point, computes K distances, and accumulates into the centroid accumulators in the same way at every iteration.

Measurement protocol.

Each configuration is repeated 5 times; mean and standard deviation are reported. For OMP for strong scaling the input size is $N \geq 500k$ and $N \geq 100k$ for weak scaling. This N are chosen because they are large enough, in a way that time is dominated by computation, rather than OS scheduling noise.

Following the methodology prescribed in the course, the serial baseline is *not* a separate serial executable, but the parallel program run with `OMP_NUM_THREADS=1`. The metrics I used are the Speedup and efficiency.

$$S(p) = T(1)/T(p) \quad E(p) = S(p)/p$$

For CUDA instead, I used the Throughput and Speedup (relative to the performance of the most efficient CPU implementation).

$$T(N) = \frac{\text{Number of operations}}{\text{Wall-clock time}} = \frac{(N * K * D * \text{Iterations})}{T_{\text{wall}}} \quad S = \frac{T_{\text{CPU}}}{T_{\text{GPU}}}$$

3 OpenMP Parallelization

3.1 Version 1

Since I scale N , I apply fine-grained partitioning of the points also, for large N and fixed D , assigning each thread to a contiguous block of N/p points yields an embarrassingly parallel workload.

In this version, `classify` and `update_centroids` are kept as separate functions and parallelized independently. Both phases use a regular partition pattern over the N points; the update phase additionally requires a reduce step to merge per-thread partial sums into the global centroid accumulators.

Regarding the memory access pattern, `data[]` is stored *row-major*, and inside `classify` and `update_centroids`, `data` is accessed always *sequentially per point* (maximizing cache hits), while `centroids[]` and all the other auxiliary structures are small enough to reside in L1/L2 for all threads simultaneously ($K \times D \times 4 \text{ B} = 1280 \text{ B}$). To avoid one atomic operation per point per dimension, each thread maintains a private accumulator `local_nc[]` and merges at the end of each iteration. The merge cost is $K \times D$ atomics per thread, negligible compared to $N \times K \times D$.

Additional optimizations include the use of `memset()`, SIMD (effective only with `-O3`) and a reduced operation count in `sqdist()`.

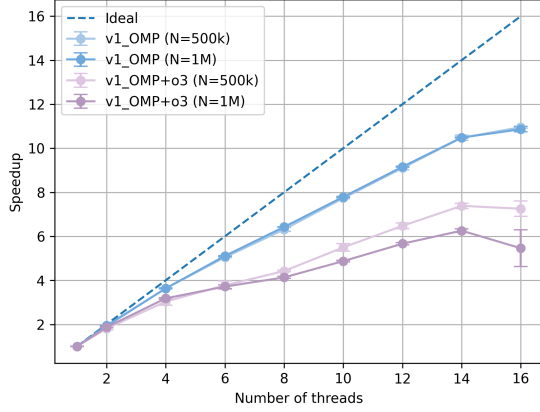


Figure 2: Strong scaling, Version 1.

As first step I analysed $v1_OMP$ ($N = 500k$) strong scaling results Fig 2. The speedup curve grows nearly linearly up to $p = 4$ ($3.63\times$, efficiency 91%), but the strange part is between $p = 8$ and $p = 16$. SMT (= Simultaneous Multithreading) region yields speedups from $6.31\times$ to $10.95\times$ respectively. The resulting gain from SMT alone is roughly 73%, which is notably higher than the 15/30% typically reported in the literature. A possible explanation is that the scalar code exhibits a high stall rate (maybe due to floating-point latency), leaving sufficient idle execution slots for the second hardware thread; however, this remains a hypothesis and would require hardware performance counter analysis to confirm.

Running the same code with `-O3` ($v1_OMP + o3$) reveals the remaining potential and the effect of SIMD vect. The single-thread improvement is substantial ($61.674s \rightarrow 3.423s$ for $N = 500K$, an $18.0\times$ acceleration). However, as we could expect, parallel speedup *degrades* more easily.

A plausible explanation is a bottleneck shift, with scalar code the threads appear compute-bound, whereas with vectorized code they may become memory-bandwidth-bound (waiting on L3/DRAM).

This is consistent with the $v1_OMP + o3$ ($N = 1M$) vs $v1_OMP + o3$ ($N = 500k$) results. At $N = 500K$ the `data[]` array occupies $500K \times 40 \times 4B = 80MB$, fitting within the 96MB of L3, whereas at $N = 1M$ it grows to 160MB, exceeding L3 capacity and requiring DRAM accesses on every iteration. Accordingly, the `-O3` variant scales noticeably worse at $N = 1M$ Fig 2, while the means remain linear to each other Table 1.

As indirect evidence that $v1_OMP$ (without `-O3`) is likely compute-bound, the strong scaling speedups at $N = 500K$ and $N = 1M$ are nearly identical, even though the larger dataset exceeds L3 capacity. If memory bandwidth were the limiting factor, one would expect a visible degradation in scaling efficiency as the working set outgrows the cache.

Is important also to note that during all testing phases the processor was running background processes (as OS, UI, WSL), so the reported figures and derived observations carry some degree of uncertainty.

3.2 Version 2

$v2$ is equivalent to $v1$ except for the following modifications:

Fused classify and update pass. Both steps are merged into a single loop over the N points, so `data[i]` is read only once per iteration instead of twice in separate passes.

Table 1: $v1$ timing results

$N = 500k$				
	v1		v1+o3	
p	Mean(s)	Std(s)	Mean(s)	Std(s)
1	61.674	0.065	3.423	0.007
8	9.769	0.148	0.774	0.006
16	5.630	0.022	0.472	0.023
$N = 1M$				
	v1		v1+o3	
p	Mean(s)	Std(s)	Mean(s)	Std(s)
1	118.880	0.946	6.903	0.025
8	18.503	0.025	1.671	0.019
16	10.944	0.120	1.265	0.193

Pre-allocated thread-private buffers. `local_counts` and `local_nc` are allocated once per thread at the start of the parallel region and freed at the end, removing `malloc/free` calls from the loop.

Persistent parallel region. A single `omp parallel` spans all iterations, reducing thread-team creation cost from $O(2 \times \text{ITERS})$ to $O(1)$.

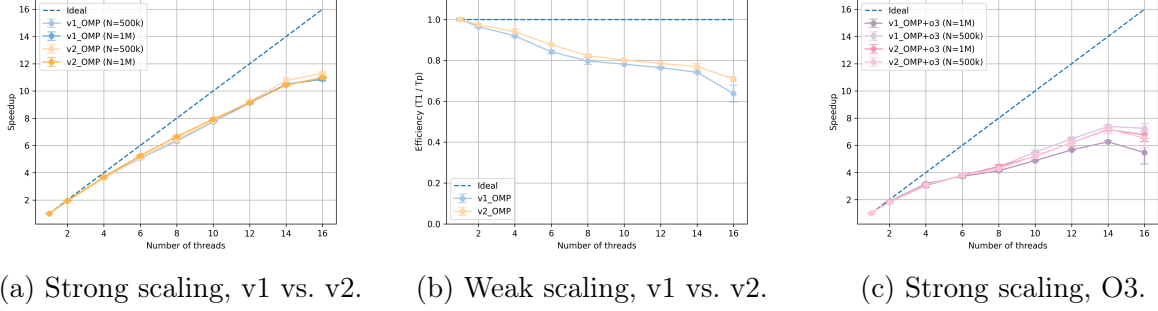


Figure 3: Scaling results for Version 2.

Table 2: *v2*, timing results

<i>p</i>	<i>v2</i>		<i>v2</i> +O3	
	Mean(s)	Std(s)	Mean(s)	Std(s)
1	56.657	0.166	3.433	0.048
8	8.816	0.312	0.801	0.018
16	5.012	0.047	0.526	0.054

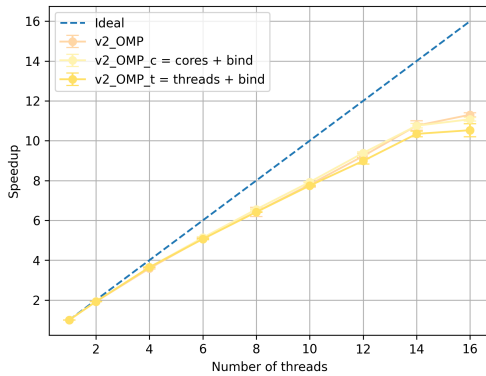


Figure 4: Strong scaling: thread binding.

As shown in Table 1 and in Table 2 ($N = 500k$), *v2* is faster than *v1* in absolute time. However, Fig 3a shows that the relative speedup curves are nearly identical between the two versions, suggesting that the optimizations do not improved inherent parallelism.

For the weak scaling experiment (Fig 3b, $N = 100k$ per p), *v2* maintains consistently higher efficiency than *v1* at every thread count. Both versions begin to lose efficiency at $p = 6$, which coincides with the point at which the total working set saturates the L3 cache ($N_{\text{total}} \times D \times 4B = 600k \times 40 \times 4B = 96 \text{ MB}$.)

Fig 3c suggests that the memory-bandwidth bottleneck observed in the *v1_o3* in Fig 2 is mitigated in *v2*. Matter of fact, *v2* + *o3* with $N = 1M$ and $N = 500k$ scale similarly regardless of whether the working set fits in L3 or not.

As a final experiment, I tested `OMP_PROC_BIND=true` with `OMP_PLACES=cores` (*v2_OMP_c*) and `OMP_PLACES=threads` (*v2_OMP_t*). Fig 4 shows the comparison becomes meaningful only at $p \in \{14, 16\}$, where the unbound configuration performs best, indicating that the OS scheduler yields the best results.

4 CUDA Parallelization

I started from a simple baseline that fuses classification and centroid accumulation into a single kernel, then iteratively improved it through two main optimizations.

Coalesced memory access. I transposed the input array from row-major ($N \times D$) to *column-major* ($D \times N$), storing it in `data_t`. With this layout, consecutive threads within a warp access `data_t[d*N + i]` in a contiguous fashion, enabling a single 128-byte coalesced transaction per warp instead of D scattered global loads.

Shared memory. Given the fixed parameters $K = 8$, $D = 40$, I allocated three dynamic shared-memory regions per block: a *read-only* copy of the centroid array (`s_centroids`, $K \times D$ floats), a *block-private accumulator* (`s_new_centroids`, $K \times D$ floats), and a *block-private count* array (`s_counts`, K integers).

The total shared memory requirement is $(2(K \times D) + K) \times 4B = 2592B$ per block, well within the 48KB default limit. Caching centroids in shared memory is particularly effective because every thread in the block reads all K centroids.

The centroid update step (division by counts) is performed on the CPU, which is negligible for $K = 8$, $D = 40$.

4.1 Tests

To estimate the optimal problem size, I targeted 100% occupancy. The RTX 4060 Ti supports up to 1536 threads per SM; with $\text{BLKDIM} = 256$ (a multiple of the warp size that also allows for flexible register allocation), one needs $1536/256 = 6$ resident blocks per SM. Filling all 34 SMs requires $34 \times 6 = 204$ blocks, corresponding to a minimum dataset size of $N_{\text{base}} = 204 \times 256 = 52,224$ to fully saturate the GPU. I ran tests using exact multiples of this value, but compared to using standard round sizes ($N \in \{500k, \dots, 8M\}$), it produced no measurable difference. Therefore, to remain consistent with the CPU experiments, we adopted the latter sizes.

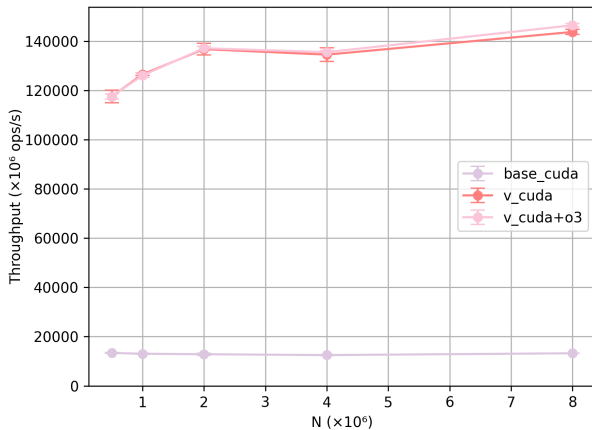


Figure 5: Throughput scaling and timing comparison across GPU versions. `base` refers to the naive GPU baseline (no coalescing, no shared memory).

Table 3: Timing results (mean $\pm$ std, seconds). N is expressed in multiples of 500k.

N	<code>base_cuda</code>		<code>v_cuda</code>		<code>v_cuda+O3</code>	
	Mean	Std	Mean	Std	Mean	Std
0.5M	1.796	0.017	0.204	0.005	0.204	0.002
1M	3.678	0.049	0.379	0.001	0.381	0.003
2M	7.472	0.206	0.702	0.012	0.700	0.004
4M	15.351	0.208	1.427	0.029	1.417	0.006
8M	29.004	0.092	2.671	0.019	2.622	0.012

As shown in Fig. 5, the throughput of `v_cuda` grows with N . A plausible explanation is that, at small N , the limited number of active blocks restricts performance. When a

warp stalls waiting for VRAM data, the SMs remain partially idle because there are not enough alternative warps to schedule in the meantime. At larger N many more blocks are in flight simultaneously, so while some warps wait on memory others continue computing. From Table 3, the `-O3` flag produces negligible improvement on the GPU (in sharp contrast to the $18\times$ gain we saw before on the CPU). This is quite expected, `nvcc` already applies aggressive optimizations to device code by default. The marginal gain at larger N may reflect slightly better host-side code for the CPU update phase and `cudaMemset/cudaMemcpy` calls. This further reinforces the conclusion that the key optimizations are memory coalescing and shared memory reuse, a conclusion that becomes even clearer when comparing `v_cuda` against the naive GPU baseline `base_cuda`.

5 CPU vs. GPU Comparison

Figure 6: CPU vs. GPU

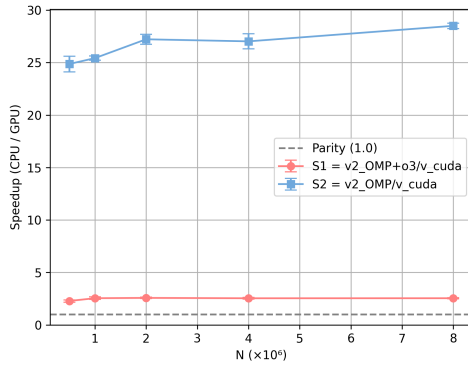


Table 4: CPU ($p = 14$) vs. GPU timing and speedup.

CPU1 = OMP_v2+o3;

CPU2 = OMP_v2(no O3).

N	CPU1 (s)	CPU2 (s)	GPU (s)	S_1	S_2
0.5M	0.526	5.012	0.204	2.28	24.87
1M	0.965	9.645	0.379	2.54	25.42
2M	1.807	19.109	0.702	2.57	27.22
4M	3.622	38.567	1.427	2.54	27.02
8M	6.790	76.139	2.671	2.54	28.50

Given the negligible difference between `v_cuda` and `v_cuda+o3`, I use `v_cuda` as the GPU reference throughout.

S_1 : GPU vs. OMP v2 +O3 at $p = 14$. As shown in Fig. 6, the speedup S_1 initially grows slowly with N , before appearing to stabilize at approximately $2.5\times$.

S_2 : GPU vs. OMP v2 (no O3) at $p = 14$. S_2 ranges from approximately $25\times$ to $28\times$ and grows with N . The ratio $S_2/S_1 \approx 10\text{--}11\times$ broadly reflects the speedup that `-O3` provides to the parallel `v2_OMP` CPU cod.

Conclusion A speedup of $2.3\text{--}2.5\times$ is notable but does not represent an overwhelming advantage, OMP v2 +O3 at $p = 14$ represents an exceptionally strong CPU baseline for this specific workload. The GPU is likely to provide a more substantial advantage when D and K are large, as more work per point reduces the relative impact of memory latency (preliminary tests with $D = 500$ and $D = 1000$, not reported here due to space constraints, appear to support this). In the end I think S_2 would make the GPU appear far more advantageous, but it would not constitute a fair comparison against the best available CPU configuration.